Memory Is Architecture: A Census of KV-Cache Footprints in Open-Weight Language Models, 2023–2026

Gurkan Ozkan
Istanbul Technical University
ozkangu@itu.edu.tr

October 2026

Abstract

Practitioners size serving hardware by parameter count, but long-context and multi-sequence serving is bound by the key–value (KV) cache, whose size follows attention geometry—layers, KV heads, and head dimension—not parameter count. We census 2331 open-weight decoder checkpoints released on the Hugging Face Hub since January 2023 (2068 after deduplicating base/instruct twins, 292 distinct attention geometries; 33 publishers), computing KV bytes per token from every public model config that exposes its attention geometry (9 do not). We ask four questions. Did KV cost per parameter fall over time (RQ1)? Do parameter count or attention mechanisms predict the KV footprint (RQ2)? How much context and concurrency fit fixed memory budgets (RQ3)? Do analytic estimates match the caches a reference implementation actually allocates (RQ4)? Median KV cost per parameter fell to 0.413 of its 2023 value under the pre-registered checkpoint-family unit, to 0.071 under one-row-per-geometry deduplication, but to 1.061 once multimodal checkpoints—whose parameter counts include vision towers—are excluded; H1 is supported on the first two units and refuted on the last. Parameters alone predict weakly ($R^2 = 0.398$) while geometry and mechanism indicators alone predict most of the variance ($R^2 = 0.792$; H2: weak, robust to the deduplication unit). Mixture-of-experts checkpoints carry 0.018 of the dense per-parameter KV cost (H3: supported, robust). The KV-parity context grew 2.2× under the pre-registered unit but 11.0× under geometry deduplication, so H4's refutation is unit-sensitive. Analytic estimates match 6 of 7 validation rows within 1%, across 6 attention classes. Memory is architecture: read the config, not the parameter count.

1 Introduction

Practitioners size serving hardware by parameter count: count parameters, multiply by bytes per weight, buy that much memory. This habit breaks down for long-context and multi-sequence serving. There, the binding constraint is the key–value (KV) cache that attention layers append to at every decoded token. Serving systems already treat it as the scarce resource: PagedAttention exists because per-request KV memory dominates and fragments GPU memory [17], DistServe replicates the KV cache when it splits prefill from decoding [33], and FlashAttention restructures the kernel around memory traffic [5].

The KV cache does not scale with parameter count. Its size follows three architectural choices. *Attention geometry*: a full-attention layer stores keys and values for $2 \times n_{\text{kv heads}} \times d_{\text{head}}$ elements per token, so layers, KV heads, and head dimension set the marginal bytes. *Attention scope*: a sliding-window layer retains only its window, not the whole context [14]. *State type*: recurrent and linear-attention layers carry a fixed-size state instead of a growing cache [12, 16], and multi-head latent attention compresses keys and values into a low-rank latent [7]. Grouped-query and multi-query attention cut KV heads directly [23, 2], and cross-layer sharing stores one cache for many layers [26]. Two checkpoints with equal parameter counts can therefore differ by an order of magnitude in KV bytes per token. On unified-memory edge devices, where weights and KV compete for one pool of memory, this difference decides how many concurrent long contexts fit; measured runtime comparisons on such hardware already rank models by KV and prompt-cache behavior [22].

Which of these designs the field actually ships is an empirical question, and the answer changes what hardware practitioners should buy. Answering it needs a census: open weights so the config is public,

published configs so geometry is exact, and release dates so trends are datable. We built one from the Hugging Face Hub, restricted to original publishers and decoder-only language models released since January 2023. The census covers 2331 checkpoints from 33 publishers; after deduplicating base/instruct twins that share geometry, 2068 remain. We skipped 237 gated repositories whose configs are not publicly downloadable and report this as a coverage limitation. The deduplicated set spans full attention (MHA, MQA, GQA), sliding-window and local/global mixes, latent attention, recurrent hybrids (Mamba, RWKV, gated DeltaNet), and cross-layer KV sharing. Table 1 summarizes the census by year.

We ask four research questions and report four pre-registered verdicts (thresholds fixed before trend inspection):

- **RQ1 (trend).** How did KV bytes per token, and KV bytes per token per billion parameters, evolve from 2023 to 2026? *H1: the median per-parameter KV cost fell substantially* — verdict supported, the pooled 2025–2026 median is 0.413 of the 2023 median.
- **RQ2 (mechanisms).** How much cross-model variance do GQA ratio, head dimension, depth, window share, recurrent share, latent attention, and cross-layer sharing explain, relative to parameter count? *H2: parameter count is a weak predictor* — verdict weak, $R^2 = 0.398$ for parameters alone versus $R^2 = 0.792$ for geometry and mechanisms alone. *H3: mixture-of-experts (MoE) models have much lower KV per total parameter* — verdict supported, the MoE median is 0.018 of the dense median.
- **RQ3 (capacity).** Under fixed memory budgets and weight precisions, what context and concurrency can each model serve? We define the *KV-parity context* T^*: the context length at which one sequence's KV bytes equal the model's weight bytes. *H4: the KV-parity context grew by orders of magnitude* — verdict refuted, the median grew only 2.2×.
- **RQ4 (validity).** Do analytic estimates match the cache tensors a reference implementation (HF `transformers`) actually allocates? Validation covers 7 models in 6 attention classes; 6 rows agree within 1%, and we explain every larger gap.

Our contributions:

- A census of 2331 open-weight checkpoints with per-checkpoint KV accounting that covers every attention class we found on the Hub, released as data, code, and reproducible pipeline.
- Population-level trend and regression results: per-parameter KV cost is falling, but it is set by architecture, not size; MoE amplifies this separation between parameters and memory.
- A capacity model that turns each config into context and concurrency limits under stated budgets and precisions, with the KV-parity context as a single comparable number per model.
- A validation experiment checking analytic bytes against allocated cache tensors, including the cases where implementations diverge from the theoretical minimum.
- An honest negative result: H4 is refuted. The median KV-parity context grew only 2.2×, not by orders of magnitude. We report the refutation as a first-class finding, not a failure.

The one-sentence thesis is *memory is architecture*: to know what a model costs to serve, read its config, not its parameter count.

2 Related Work

The mechanisms we count were proposed one paper at a time; we group them by theme and state what each changes in the cache.

KV-head reduction. Multi-query attention traces slow decoding to repeated key–value loads and shares one KV head across all query heads [23]. Grouped-query attention interpolates between multi-head and multi-query and is cheap to uptrain from a multi-head checkpoint [2]. Multi-head latent attention compresses keys and values into a low-rank latent plus a small rope component; DeepSeek-V2 reports a large KV-cache reduction and higher generation throughput [7]. These designs cut the KV-head term of the geometry without shrinking the model.

Attention scope and sparsity. Mistral 7B pairs grouped-query attention with a sliding window, bounding each layer's cache by the window rather than the context [14]. StreamingLLM shows window attention fails once text exceeds the cache and that retaining initial-token sinks restores stability over millions of tokens [29]. H2O evicts dynamically, keeping recent tokens plus a small set of heavy hitters [32]. We count windowed layers as they ship in configs; eviction at serving time is orthogonal and outside the census.

KV compression and quantization. KIVI quantizes the KV cache to 2 bits, keys per channel and values per token [19]; KVQuant pushes long-context KV to low precision [13]; Palu projects the cache to low rank and reconstructs keys and values on the fly [3]. On the weight side, GPTQ [9] and outlier-aware Int8 [8] shrink the term the cache competes with. Our capacity model treats weight and KV precision as parameters; the census itself is BF16.

Cross-layer KV sharing. YOCO caches key–value pairs once and reuses them across layers through cross-attention [26]. Layer-condensed KV condenses the key–value cache into a subset of layers and lets the remaining layers read from it [28]. The census counts each shared cache once, as the configs declare it.

Recurrent and linear-attention state. Linear attention rewrites attention as a kernel dot product with an RNN-style constant-state update [16]; RetNet derives a retention operator with a recurrent mode [25]; RWKV trains an RNN at transformer scale [20]; Gated DeltaNet adds gating and a delta rule to the fixed state [31]. Selective state-space models make the state input-dependent [12]. Hybrids mix these with attention: Griffin interleaves gated linear recurrences with local attention [6]; Jamba interleaves Transformer, Mamba, and MoE layers [18]; Falcon-H1 runs attention and SSM branches in parallel [34]. Our accounting carries their state as a fixed, context-independent term.

Mixture-of-experts. Mixtral activates two of eight experts per token, decoupling total from active parameters [15]. DeepSeek-V2 combines the sparse-router trick with latent attention [7]. Routers add parameters without adding cache, which is why we test KV cost per total parameter against dense models as a separate hypothesis.

Long-context scaling. Rotary position embeddings give flexible sequence length to attention [24]; YaRN extends trained context windows cheaply [21]. Longer declared contexts raise the stakes of every byte per token, which is the trend our capacity analysis prices.

Model family reports. Publisher reports document the geometry shifts we measure: Llama 3 [11] releases a large dense family with long context; Qwen3 [30] spans dense and MoE sizes under one release; Gemma 3 explicitly rebalances local and global layers to contain KV memory at long context [10]; phi-4 stays dense [1]; OLMo 2 releases weights, data, and logs [27]. Each report is one family; none says what the population does.

Serving systems and edge. vLLM's PagedAttention manages KV memory in pages [17]; DistServe disaggregates prefill and decode onto separate resources [33]; FlashAttention and its successor cut attention memory traffic [5, 4]. On unified-memory Apple Silicon hardware, production-grade runtime comparisons treat KV and prompt caching as first-class metrics [22]. These systems manage whatever cache a model brings; they do not say how large that cache is across models.

Position. The works above propose mechanisms or systems and evaluate them on a handful of checkpoints. We measure, at population scale, which mechanisms publishers actually ship and what each costs in KV bytes per token, with release dates attached. To our knowledge no public dataset ties Hub release metadata to per-checkpoint KV accounting across attention classes; the census fills that gap.

3 Data and Methods

3.1 Census scope and inclusion criteria

We census open-weight, decoder-only language models published on the Hugging Face Hub between January 2023 and the census snapshot in October 2026. We keep checkpoints released by the original publisher organization (33 organizations) and exclude fine-tunes, merges, and quantized re-uploads from third-party accounts; fine-tunes and distills hosted by the original publisher themselves are kept (a documented decision, with its consequences visible in the deduplication analysis below). Multimodal models enter through their text backbone: we parse the language-model sub-configuration and ignore vision and audio towers. Their *parameter counts*, however, come from repository safetensors metadata and therefore include the vision tower (flagged per row in the `notes` column; 169 deduplicated rows): per-parameter KV statistics for these rows are deflated, which is why RQ1 reports a multimodal-excluded robustness cut, while capacity computations correctly use the full weight footprint. The census holds 2331 checkpoint rows — 263 released in 2023, 670 in 2024, 1044 in 2025, and 354 in 2026 to date. Every `config.json` is snapshotted and all Hub API responses are cached on disk, so the pipeline re-runs offline and deterministically.

Selection effects are counted, not hidden. Gated repositories whose `config.json` is not publicly downloadable are skipped; 237 repositories were skipped this way, which biases the census away from gated families and is reported as a selection-bias limitation. The release date is the Hub `createdAt` timestamp, a proxy that can predate or lag the effective public release. Parsing failed for 9 configurations; these are listed in `data/kv_config_errors.txt` and excluded from the census table. 104 included rows carry no parameter count (neither safetensors metadata nor a sufficient configuration); both gaps are carried as limitations. Where a count exists, it comes from Hub safetensors metadata, otherwise it is computed from the configuration; the per-row source is recorded in the `params_source` column.

3.2 KV-cache accounting conventions

All cache sizes are analytic per-sequence minima computed by `analysis/kv.py`, a library of pure functions over the snapshotted configuration dictionaries; it never loads weights. A full-attention layer caches one key and one value tensor of $n_{\mathrm{kv}} \times d_h$ elements per token, i.e. $2\, n_{\mathrm{kv}}\, d_h\, B$ bytes per token, where B is the bytes per element (2 for BF16). Grouped-query and multi-query attention are the same formula with reduced n_{kv} [23, 2]. A sliding-window layer with window W holds $\min(T, W)$ tokens at context T [14]. A multi-head latent attention layer caches only the compressed latent, $(r_{\mathrm{kv}} + d_{\mathrm{rope}})\, B$ bytes per token, where r_{kv} is the latent rank (`kv_lora_rank`) and d_{rope} the decoupled RoPE dimension (`qk_rope_head_dim`) [7]; values are decompressed on the fly, so we count the theoretical latent and measure the as-implemented cache in the validation experiment. A recurrent layer — Mamba-style state-space layers [12], linear attention [16], gated delta-rule, RWKV — holds a constant state whose size is computed from the configuration (SSM and convolution state, or a matrix state) and does not depend on T. Falcon-H1 layers carry both an attention and a recurrent cache, and each is counted. Caches shared across layers are counted once [26].

The headline outcome is the asymptotic marginal cache cost per token, $\lim_{T\to\infty} \mathrm{d}\,\mathrm{bytes}(T)/\mathrm{d}T$: only layers whose cache grows without bound (full and latent layers) contribute, since windowed layers saturate at W tokens and recurrent state is constant. Labels are honest by construction: a model is labelled `mha`, `gqa`, or `mqa` only if it is pure full attention and the computed per-token bytes equal $2\, L\, n_{\mathrm{kv}}\, d_h\, B$ exactly; any windowed, recurrent, latent, or shared component forces a descriptive label such as `gqa+swa`, `hybrid-mamba`, or `mla`; configurations whose literal fields disagree with their family's canonical semantics carry an additional `+legacycfg` marker. Family-specific parsing rules follow the upstream `transformers` configuration semantics — explicit head dimensions, per-family window flags, hybrid layer-pattern strings — cross-checked against published configurations [11, 30]. The library carries 107 unit tests against hand-computed values and 61 real configuration snapshots; an unknown architecture raises an error rather than defaulting to a guess.

3.3 Pre-registered analysis

The analysis plan (`analysis/PLAN.md`) was fixed before any trend statistic was computed, and every deviation is recorded there as a dated note. Trend and regression statistics use a deduplicated set: the `dedup_group` key hashes the publisher organization, the name stem (base/instruct/chat and similar suffixes stripped), and the attention geometry, and the representative is the earliest-released member, ties broken by repository id. This collapses base/instruct twins but not same-geometry fine-tunes with distinct names: it leaves 2068 rows in 2068 groups realizing only 292 distinct geometries (482 organization–geometry combinations), with one geometry alone accounting for 260 rows. We therefore report the pre-registered family-level unit as primary and a strict one-row-per-geometry unit as the robustness view throughout RQ1–RQ4. RQ1 reports per-year medians and interquartile ranges of the two primary outcomes and Spearman rank correlations against release date, with pre-registered support thresholds. RQ2 regresses $\log_{10}$ bytes/token on $\log_{10}$ parameters by OLS, with 95% confidence intervals from 1000 bootstrap resamples at seed 0, and fits a mechanism regression adding GQA ratio, head dimension, depth, window share, recurrent share, a latent indicator, and a shared-cache indicator, reported as standardized coefficients with variance inflation factors. The MoE-versus-dense comparison uses a two-sided Mann-Whitney U test with a rank-biserial effect size. Capacity computations use all rows: each checkpoint is its own capacity point.

3.4 Capacity model

For every checkpoint we solve a one-dimensional memory budget: usable KV pool = budget − weight bytes − runtime overhead. Budgets are 24, 80, and 128 GiB (2^{30} bytes), matching common accelerator and unified-memory tiers. Weights occupy 2 bytes per parameter (BF16), 1 byte (FP8), or 4.5 bits per parameter under 4-bit quantization, the effective figure including 16 group-scale bits per 32-element group [9]. The KV cache is priced at 2 or 1 byte per element (BF16 or FP8) [19]; we do not model allocator fragmentation, which serving systems manage separately [17]. Runtime overhead is a free parameter: the full grid over 0, 2, 6, and 12 GiB is computed and exported with the results (every budget, weight precision, and KV precision combination), the paper shows a sensitivity table at 128 GiB, and the headline tables use 6 GiB. Because bytes(T) is monotone piecewise linear with breakpoints at the window sizes, the maximum token count under the pool and the KV-parity context T^* — the length at which one sequence's cache equals the BF16 weight bytes — are solved exactly on each linear segment. The plan pre-registered a log-spaced grid with interpolation; the exact solve is a strict refinement of the same model, recorded as a dated deviation.

3.5 Validation against allocated caches

Analytic minima are meaningless if the reference stack allocates something else. For RQ4 we instantiate reduced models directly from the snapshotted configurations with `from_config` (random initialization, no weight downloads, CPU only). Depth is cut to the smallest value that preserves one full period of the layer-type pattern, the vocabulary is shrunk, and per-layer attention geometry — KV heads, head dimensions, latent ranks, window sizes — is kept exact. We prefill T tokens, with $T > W$ for windowed models, and sum the bytes of every cache tensor the returned cache object holds: keys and values, convolution and SSM states. The analytic side is evaluated on the same reduced configuration in float32. The relative error is $|\text{measured} - \text{analytic}|/\text{analytic}$; every row above 1% must carry an explanation, and the explanation itself is a reported finding. The experiment covers 7 models spanning 6 attention classes, with 6 rows within 1%.

4 Results (RQ1–RQ4)

4.1 RQ1: per-token KV cost fell every year; per-parameter cost fell with a 2025 rebound

Table 1 and Figure 1 summarize the deduplicated census of 2068 geometry groups. The median marginal KV cost in BF16 fell in every release year: 327680 bytes per token in 2023, 131072 in 2024, 70272 in 2025, and 36864 in 2026. Spearman rank correlations against release date confirm a decline on both outcomes: $\rho = -0.213$ ($p = 1.3e-22$) for KV bytes per token and $\rho = -0.298$ ($p = 1.1e-41$) for KV per parameter.

Table 1: Census overview. Rows are checkpoints; family-dedup keeps one row per publisher family with identical geometry, geo-dedup one row per distinct attention geometry.

year	rows	family	geo	med. KV/tok	IQR	B/tok/10^9p	geo B/tok/10^9p
2023	263	243	50	327680	487424	72087.7	61170.6
2024	670	563	78	131072	240640	25720.2	14800.3
2025	1044	949	104	70272	172032	76468.6	9238.2
2026	354	313	60	36864	135168	7374.3	861.7

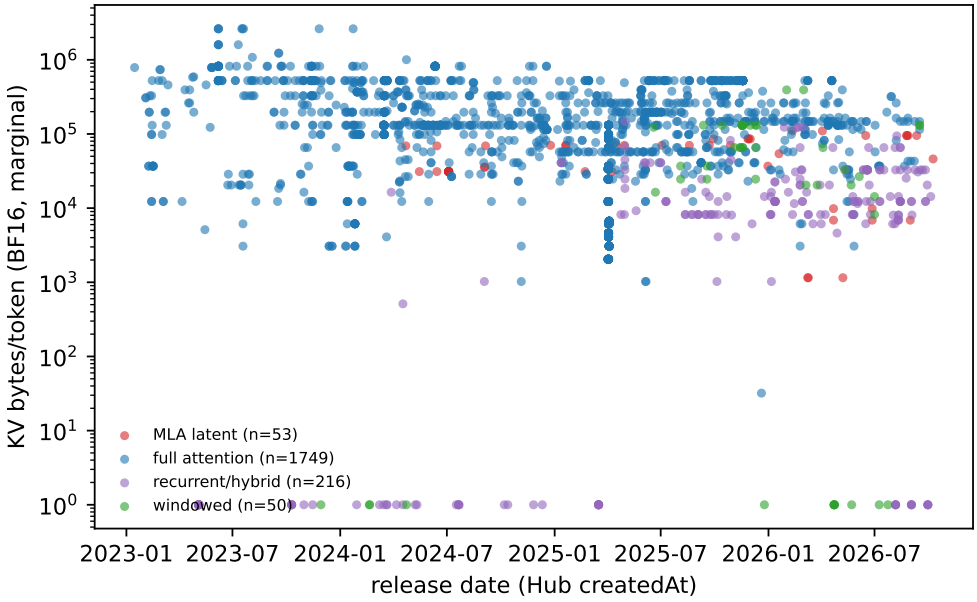


Figure 1: Marginal KV bytes per token (BF16) against release date for the deduplicated census, log-scale y axis; color encodes the coarse attention class.

The correlations are weak in magnitude because the within-year spread dwarfs the between-year shift; in Figure 1 the full-attention, windowed, recurrent/hybrid, and MLA-latent classes overlap across orders of magnitude at every date. The decline tracks the diffusion of cost-saving designs rather than a single event: KV-head reduction cuts the per-token cost at fixed width [23, 2], sliding-window layers bound the cache [14], and latent compression shrinks it further [7]; Figure 1 shows windowed and recurrent/hybrid points becoming denser at later dates.

The per-parameter picture depends on the counting unit, and we report all of them. Under the pre-registered checkpoint-family unit, median KV bytes per token per 10^9 parameters dropped from 72087.7 in 2023 to 25720.2 in 2024, rebounded to 76468.6 in 2025—above the 2023 level—and fell to 7374.3 in 2026. The rebound is an artifact of duplicated geometry, not an architectural regression: the 2068 family-level groups realize only 292 distinct attention geometries, because same-geometry fine-tunes with distinct names survive family-level deduplication (worst case: 260 rows of a single 2023-era geometry, 143 of them released in 2025 or later). Under one-row-per-geometry deduplication the series is monotone: 61170.6, 14800.3, 9238.2, 861.7 (Table 1), and Spearman ρ strengthens to -0.407 ($p = 3.5e-12$).

Against the pre-registered threshold, H1 is supported on the pre-registered unit: the pooled 2025–2026 median KV-per-parameter cost is 0.413× the 2023 median. Under geometry deduplication the ratio is 0.071, also below the registered support level of 0.75. One robustness cut reverses the verdict: multimodal checkpoints (169 of the dedup set) carry parameter counts that include their vision towers while their KV accounting covers only the language backbone, deflating their per-parameter KV cost; excluding them yields a pooled ratio of 1.061, above 0.75. H1 is therefore supported under the pre-registered analysis and under geometry deduplication, but is sensitive to how multimodal parameter counts are handled; we flag this rather than pick the flattering cut.

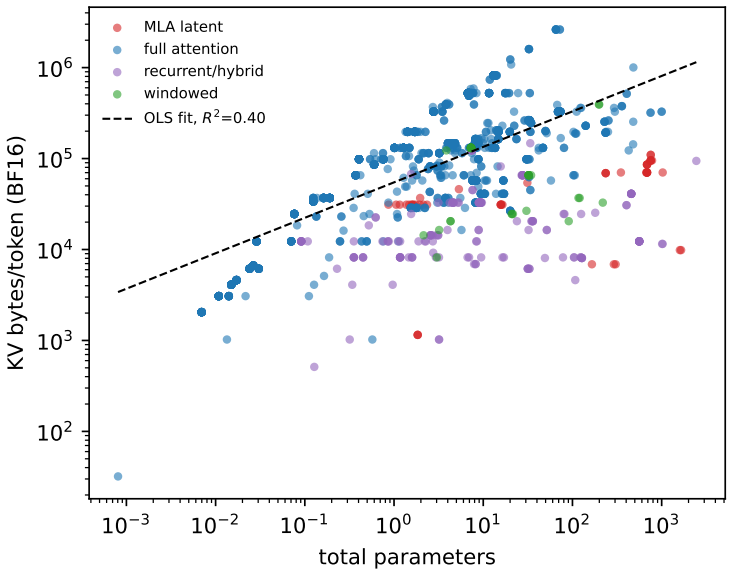


Figure 2: KV bytes per token against total parameters (log–log) with the OLS fit; color encodes attention class.

Table 2: Standardized coefficients of the mechanism regression on $\log_{10}$ KV bytes/token.

predictor	β (std.)	VIF
total params ($\log_{10}$)	0.601	7.3
GQA ratio ($\log(1+r)$)	-0.617	2.0
head dim	0.245	2.3
depth ($\log(1+L)$)	0.433	5.6
window layer share	-0.120	1.1
recurrent layer share	-0.315	1.1
MLA latent (0/1)	-0.349	1.2
cross-layer shared KV (0/1)	-0.014	1.0
N	1930 (cross-layer shared: $n = 2$)	
R^2 (full)		0.841

4.2 RQ2: parameter count is a weak predictor; geometry explains most variance

Figure 2 regresses $\log_{10}$ KV bytes per token on $\log_{10}$ total parameters. The slope is 0.390 (95% CI 0.363–0.416) with $R^2 = 0.398$ (95% CI 0.350–0.448, 1000 bootstrap resamples), so H2 is weak by the pre-registered criterion ($R^2 < 0.5$): parameters alone leave most variance unexplained. The same regression on one row per geometry gives $R^2 = 0.254$, so the weakness is not an artifact of duplicated geometries. Attention geometry alone—GQA ratio (attention heads over KV heads; imputed to 1 where KV heads are absent or equal to attention heads), head dimension, depth, window share, recurrent share, and the MLA and cross-layer-sharing indicators—predicts KV cost with $R^2 = 0.792$; adding log parameters raises it to 0.841 (parameters alone: 0.398). Two caveats: the outcome is by construction an exact function of the geometry predictors, so the geometry R^2 is only informative as a contrast with parameter count, and the full model still adds 0.841 versus 0.792. The regression uses $N = 1930$ groups after excluding 97 groups without parameter counts and 67 zero-marginal-KV groups, whose asymptotic marginal cost is zero because every cache-bearing layer is windowed or recurrent (26 groups lack both).

Table 2 reports fully standardized coefficients. Every sign matches naive per-layer accounting: KV-head reduction through GQA ($\beta = -0.617$), latent compression ($\beta = -0.349$, consistent with [7]), recurrent state replacing KV ($\beta = -0.315$), and windowed layers ($\beta = -0.120$) all lower predicted KV cost, while depth

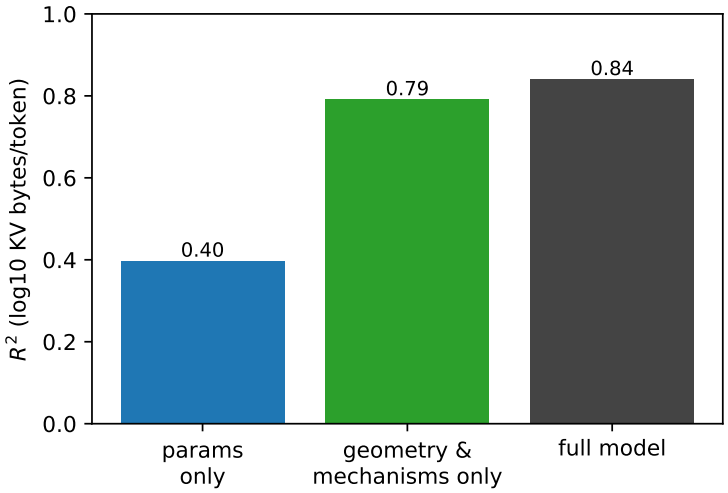


Figure 3: Variance explained (R^2) of $\log_{10}$ KV bytes/token by parameter count alone, by attention geometry and mechanisms alone, and by the full model. Parameter count is a weak predictor compared with geometry.

raises it ($\beta = 0.433$): a model that spends its budget on more layers buys KV with them. The cross-layer-sharing indicator is estimated from only 2 positive observations and its coefficient ($\beta = -0.014$) should not be read as evidence; the mechanism is grounded instead in the accounting itself [26, 28]. Publisher behavior mirrors these mechanisms: Gemma 3 re-architected its attention layout specifically to curb KV growth by shortening local spans and thinning global layers [10]. Because the predictors collinearly encode one architecture (VIFs up to 7.3 for parameters and 5.6 for depth, Table 2), we read the variance decomposition as the mechanism evidence, not the individual magnitudes.

MoE models carry far less KV per total parameter, confirming H3 (supported). Median KV per 10^9 parameters is 62940.6 for dense models ($N = 1695$) against 1110.5 for MoE models ($N = 235$): a ratio of 0.018, far below the registered 0.5 threshold (Mann–Whitney $p = 2.7e-99$; rank-biserial -0.850 for the MoE group, negative meaning MoE values rank lower). The ratio is 0.024 under geometry deduplication. Experts add parameters without adding per-token attention cache [15, 7], and recent open families ship dense and MoE variants on the same attention stack [30]. Sizing serving hardware by total parameter count therefore overstates the memory cost of sparse models.

4.3 RQ3: the KV-parity boundary moved, but not by orders of magnitude

We define the KV-parity context T^* as the context length at which one sequence's KV bytes equal its BF16 weight bytes; beyond T^* the cache, not the weights, dominates memory. Figure 4 and Table 3 give the capacity view under fixed budgets. Three regimes coexist. Small full-context models serve tens of thousands of KV tokens at 24 GiB and hundreds of thousands at 128 GiB. The largest recent open models enter the table only at 128 GiB, several of them only once weights drop to 4-bit—`gpt-oss-120b` reaches 1.8M total KV tokens there—and 74 dedup groups with known parameters do not fit 128 GiB even at 4-bit weights once the modeled 6 GiB runtime overhead is applied (64 of them exceed the budget on weights alone). Concurrency tells the same story: at 128 GiB with BF16 weights and KV, the median model that fits at all serves 30 concurrent 32k contexts but only 8 concurrent 128k contexts, and 12.6% of groups cannot hold even one 128k context. Table 4 varies the runtime overhead across the pre-registered 0–12 GiB grid; the full grid (budgets × weight precisions × KV precisions × overheads) is exported with the results. At the other extreme, `starcoder2-3b` never saturates: every layer is windowed, so its cache is bounded and total capacity is unlimited. How much precision buys depends on the weight share of the budget. `OLMo-7B` serves 11k tokens at 24 GiB with BF16 weights and 24k with FP8 weights, because freed weight bytes flow straight into the KV pool; at 128 GiB the same model gains little (224k versus 237k) because its budget is already KV-dominant. For the largest models, weight quantization [9] and KV quantization [19, 13] decide whether

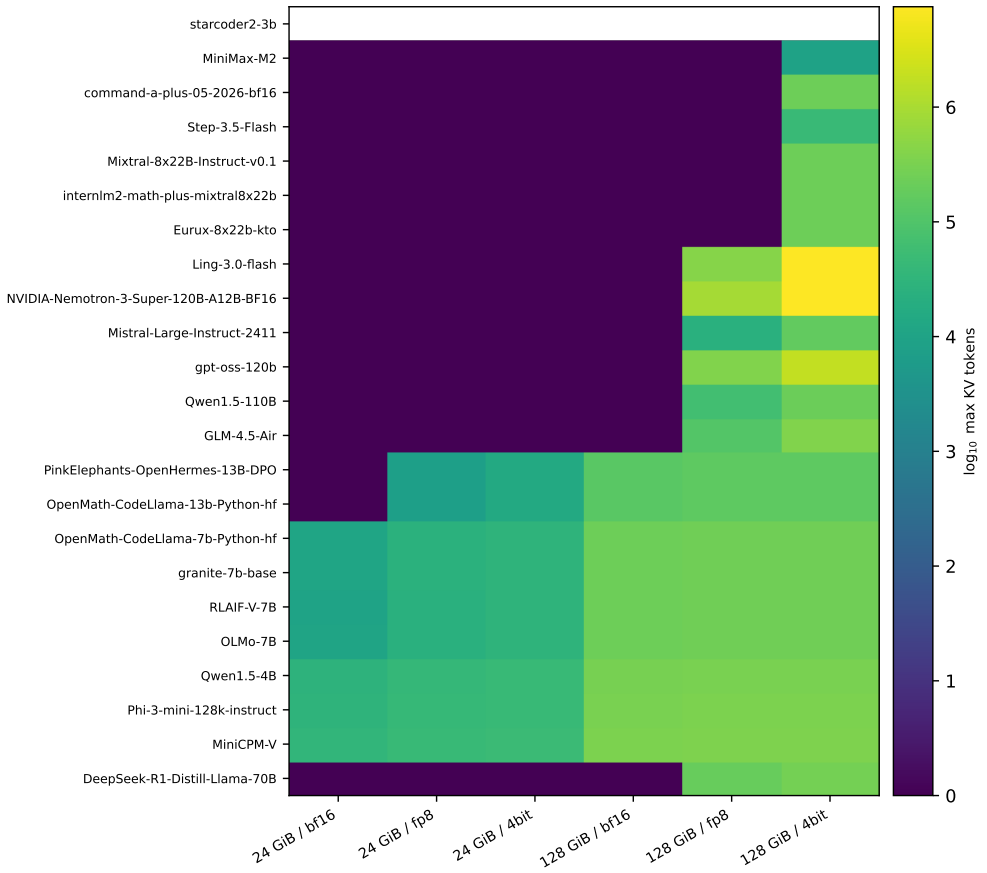


Figure 4: Maximum total KV tokens (color: $\log_{10}$) across memory budgets and weight precisions, KV in BF16, 6 GiB runtime overhead; rows follow the objective selection rule documented in the analysis code.

any cache fits at all. This is the regime PagedAttention was built for: KV memory, not parameters, limits batch size [17], and on unified-memory devices it decides how many long contexts coexist [22].

The parity trend (Figure 5) is the honest counterweight to the capacity headline. Median T^* moved from 26753 tokens in 2023 to 72329 in 2024, fell back to 26155 in 2025—slightly below the 2023 median—and jumped to 246064 in 2026. The share of models that are KV-dominant at 32k context ($T^* < 32$k) fell from 70.7% in 2023 to 8.1% in 2026, but not monotonically: it rebounded to 56.2% in 2025, driven by the same 2025 cohort composition as in RQ1. A small and growing fraction of releases have no parity point at all—1.0% in 2023 versus 6.5% in 2026—because their caches are bounded (fully windowed or recurrent) and saturate below the weight bytes; we censor them to infinity and exclude them from the medians. Against the pre-registered claim, H4 is refuted on the pre-registered unit: pooled 2025–2026 median T^* is 2.2× the 2023 median, far from the registered 10×. The verdict is unit-sensitive: under geometry deduplication the yearly medians (32412, 122194, 180277, 1153276) give a pooled ratio of 11.0×, which crosses the registered threshold. The honest summary is that parity contexts grew several-fold to an order of magnitude depending on the counting unit—substantially, but not by the orders of magnitude the hypothesis claimed—while context demand grew apace [21].

4.4 RQ4: analytic accounting matches measured caches

The validation experiment instantiates reduced-depth, randomly initialized models from the snapshotted configs and measures the bytes of every cache tensor after prefill. It covers 7 models spanning 6 attention classes (mha, gqa, windowed GQA hybrids, MLA, hybrid-Mamba, pure recurrent), and 6 of the rows agree

Table 3: Maximum KV tokens at 6 GiB runtime overhead, KV in BF16; '–' means the weights alone exceed the budget, ∞ means the cache saturates (bounded window/state).

model	24/BF16	24/FP8	24/4b	128/BF16	128/FP8	128/4b
starcoder2-3b	∞	∞	∞	∞	∞	∞
MiniMax-M2	–	–	–	–	–	9k
command-a-plus-05-2026-bf16	–	–	–	–	–	230k
Step-3.5-Flash	–	–	–	–	–	47k
Mixtral-8x22B-Instruct-v0.1	–	–	–	–	–	226k
internlm2-math-plus-mixtral8x22b	–	–	–	–	–	226k
Eurux-8x22b-kto	–	–	–	–	–	226k
Ling-3.0-flash	–	–	–	–	431k	7.3M
NVIDIA-Nemotron-3-Super-120B-A12B-BF16	–	–	–	–	891k	7.5M
Mistral-Large-Instruct-2411	–	–	–	–	23k	172k
gpt-oss-120b	–	–	–	–	384k	1.8M
Qwen1.5-110B	–	–	–	–	60k	209k
GLM-4.5-Air	–	–	–	–	109k	365k
PinkElephants-OpenHermes-13B-DPO	–	8k	15k	128k	144k	151k
OpenMath-CodeLlama-13b-Python-hf	–	8k	15k	128k	144k	151k
OpenMath-CodeLlama-7b-Python-hf	11k	24k	30k	224k	237k	243k
granite-7b-base	11k	24k	30k	224k	237k	243k
RLAIF-V-7B	10k	23k	29k	223k	236k	242k
OLMo-7B	11k	24k	29k	224k	237k	242k
Qwen1.5-4B	28k	38k	42k	301k	310k	314k
Phi-3-mini-128k-instruct	30k	39k	44k	314k	323k	328k
MiniCPM-V	34k	43k	47k	337k	346k	350k
DeepSeek-R1-Distill-Llama-70B	–	–	–	–	184k	279k

with the analytic accounting within 1%, meeting the pre-registered success criteria (Figure 6, Table 5). Three implementation findings explain every nonzero gap. First, `transformers` 5.19 enforces sliding windows in the cache itself, not only in the attention mask: once a window saturates, the implementation truncates the stored tensor and retains one position fewer than the analytic $\min(T, W)$, which accounts for the small negative deviations on the two windowed probes. Second, the MLA implementation caches the compressed latent—the low-rank plus rope dimensions per token—and never materializes decompressed keys and values, so the theoretical latent accounting matches exactly [7]. Third, recurrent layers pad their convolution state to the full kernel width, where the analytic model charges the theoretical minimum of stored inputs; this conv-state padding is the only deviation above 1% (the pure-Mamba probe) and also explains the sub-1% residual on the hybrid probe [12]. The experiment runs in float32 and compares against float32 analytic bytes; real BF16 serving halves both sides of the comparison, so the relative agreement carries over but the absolute byte counts do not.

5 Discussion

Memory is architecture. The census supports the title claim mechanically. Parameter count alone explains 0.398 of the variance in $\log_{10}$ KV bytes per token on the deduplicated set — a weak predictor, as H2 anticipated — while attention geometry and mechanism indicators alone reach 0.792, and the combined model 0.841 (Table 2). The dominant mechanisms are discrete, inspectable design knobs, not scaling accidents: sharing KV heads across query heads [23, 2], bounding the cache with sliding windows [29, 14], compressing keys and values into latents [7], and sharing one cache across layers [26, 28]. Because these are choices, equal-parameter models diverge widely: the dense and MoE medians sit at 62940.6 versus 1110.5 bytes per token per 10^9 parameters. A deployment that sizes memory by parameter count is therefore wrong in both directions, and the error lands on the constraint that binds first [17].

The 2025 bump is composition, mostly duplicated geometry. Median per-parameter KV cost under the pre-registered unit fell from 72087.7 (2023) to 25720.2 (2024), rose to 76468.6 in 2025, then fell to 7374.3 in 2026 (Table 1). Two composition effects, not architecture, drive the rise. First, fine-tune series that reuse a 2023-era geometry with new names survive family-level deduplication, and 143 such rows from the single most-duplicated geometry landed in 2025 or later; collapsing every geometry to one row makes the series monotone (61170.6 to 14800.3, 9238.2, 861.7). Second, the remaining 2025 cohort skews small: absolute KV per token fell (131072 to 70272) while per-parameter KV rose, the signature of small checkpoints

Table 4: Overhead sensitivity: maximum KV tokens at 128 GiB, BF16 weights and KV, for runtime overheads 0/2/6/12 GiB.

model	0 GiB	2 GiB	6 GiB	12 GiB
starcoder2-3b	–	–	–	–
MiniMax-M2	–	–	–	–
command-a-plus-05-2026-bf16	–	–	–	–
Step-3.5-Flash	–	–	–	–
Mixtral-8x22B-Instruct-v0.1	–	–	–	–
internlm2-math-plus-mixtral8x22b	–	–	–	–
Eurux-8x22b-kto	–	–	–	–
Ling-3.0-flash	–	–	–	–
NVIDIA-Nemotron-3-Super-120B-A12B-BF16	–	–	–	–
Mistral-Large-Instruct-2411	–	–	–	–
gpt-oss-120b	–	–	–	–
Qwen1.5-110B	–	–	–	–
GLM-4.5-Air	–	–	–	–
PinkElephants-OpenHermes-13B-DPO	–	–	–	–
OpenMath-CodeLlama-13b-Python-hf	–	–	–	–
OpenMath-CodeLlama-7b-Python-hf	–	–	–	–
granite-7b-base	–	–	–	–
RLAIF-V-7B	–	–	–	–
OLMo-7B	–	–	–	–
Qwen1.5-4B	–	–	–	–
Phi-3-mini-128k-instruct	–	–	–	–
MiniCPM-V	–	–	–	–
DeepSeek-R1-Distill-Llama-70B	–	–	–	–

whose attention stacks stay proportionally large — and of multimodal checkpoints whose parameter counts include vision towers. Across checkpoints the pooled trend stays negative under every counting unit (family: $\rho = -0.298$, $p = 1.1e-41$; geometry: $\rho = -0.407$). The planning lesson is that a yearly fleet median tracks what publishers ship that year, deduplication unit included; in a year dominated by small releases and re-released fine-tunes, per-parameter heuristics will under-provision KV memory even while per-token architecture improves.

MoE dilutes KV per parameter, not KV per sequence. H3 was supported with the strongest effect in the census: MoE median 1110.5 versus dense 62940.6 bytes per token per 10^9 parameters (rank-biserial -0.850, p =2.7e-99). The mechanism is dilution: experts add weights without adding attention layers, so the cache is set by the shared trunk [15, 7]. Dilution is not cheapness. Per-sequence KV bytes are unchanged by expert count, while total weights crowd the memory pool: 74 deduplicated models with known parameters exceed 128 GiB at 4-bit weights under the modeled 6 GiB runtime overhead and admit no KV at all (64 of them on weights alone; Table 3). On unified-memory edge devices, where weights and KV share one pool, this asymmetry decides what can be served at all [22].

Why H4 failed. Absolute KV per token fell across the window — the median dropped from 327680 (2023) to 36864 (2026) — but the KV-parity context is a ratio of weight bytes to KV bytes, so it moves only when KV falls *relative to* weights. Both moved at once. Window-heavy and latent designs, and above all MoE weight growth, pushed parity out or removed it: 6.5% of the 2026 fleet has bounded caches and never reaches parity, and the share of models whose KV outgrows weights before 32k context fell from 70.7% (2023) to 8.1% (2026). Against this, the 2025 small-model mass pulled the yearly median back to 26155 tokens, below the 2023 median of 26753. The pooled 2025–2026 median reached only 2.2 times the 2023 value under the pre-registered unit, far short of the tenfold threshold, so H4 is refuted there — but under geometry deduplication the same statistic is 11.0× and crosses the threshold (Fig. 5). The direction of the hypothesis was right; its magnitude depends on the counting unit, because parity inherits weight-byte growth and release mix, not KV savings alone.

Capacity levers. Under fixed budgets, geometry decides which lever pays. At 24 GiB, small dense checkpoints leave most of the pool for KV; at 128 GiB the constraint flips to weights, where 74 models fit nothing even at 4-bit (Table 3, Fig. 4). Weight quantization [9, 8] and KV quantization [19, 13] expand opposite sides of the same unified pool, so the cheaper lever is a property of the specific attention geometry, not of

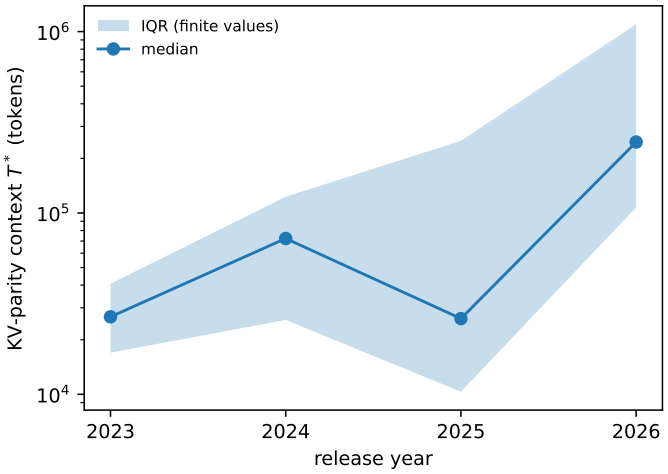


Figure 5: Median and IQR of finite KV-parity contexts T^* by release year, log-scale y axis.

parameter count.

6 Limitations

Gating and family coverage. The census covers only what is publicly downloadable without license acceptance. Gating excluded 237 repositories, and the exclusions are concentrated: every `meta-llama` checkpoint was gated at snapshot time, so the census contains no rows from that organization, and the earlier `google/gemma` catalog was skipped, leaving only the recent `gemma-4` checkpoints to represent that family. Llama-architecture checkpoints released by other included publishers are present, but the primary Llama releases are absent. Both families anchor a large share of real deployments, and the Gemma 3 report documents exactly the window-heavy, KV-saving designs this paper studies [10]. Their partial absence shifts yearly medians and the mechanism mix in directions we cannot sign from the data at hand; a Llama-only or Gemma-only census would likely sit at different medians for both KV per token and window share.

Release-date proxy and snapshot. Release dating uses the Hub `createdAt` timestamp, which measures upload, not publication. Re-uploads, staged rollouts, and repository reorganization can move a model across year boundaries, and every per-year statistic inherits that noise. A model announced in one year and uploaded in the next is dated late; variants uploaded together with their base model are dated early. The snapshot is single-point-in-time (2026-10-10): repositories were not re-crawled, deleted or renamed repos are not re-checked, and 2026 figures cover a partial year of 313 deduplicated rows, so the 2026 medians will move once the year completes.

Dropped rows and dedup heuristics. Coverage inside included repositories is imperfect. 9 configurations could not be parsed into an attention geometry (missing or non-standard fields) and were dropped. 104 rows lack parameter counts and are excluded from per-parameter statistics and from every capacity computation. The dedup key is organization, name stem, and geometry: it collapses base/instruct twins but *not* same-geometry fine-tunes with distinct names, so the 2068 pre-registered groups realize only 292 distinct geometries (260 rows share the most common one). Trend medians, p-values, and two verdicts (H1, H4) are sensitive to this unit choice, which is why every affected statistic is reported under both units. Multimodal rows (169 deduplicated rows) carry vision-tower-inflated parameter counts against language-backbone KV, deflating their per-parameter cost; RQ1 reports the multimodal-excluded cut for this reason.

Analytic values are theoretical minima. All KV figures follow from the configurations, not from measurements of deployed runtimes. Implementations can allocate more: latent attention may be cached

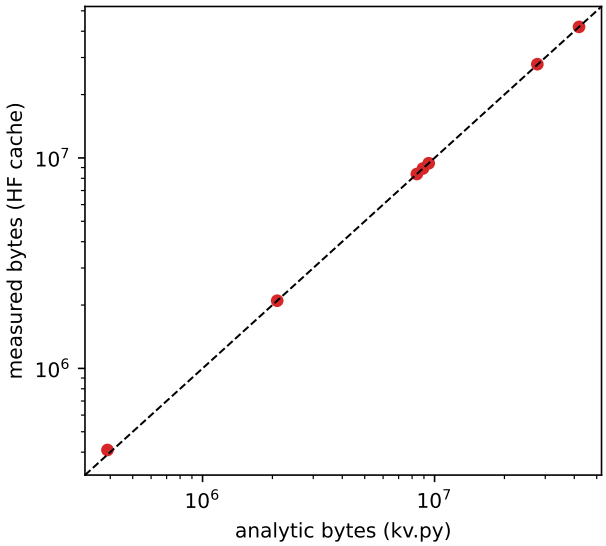


Figure 6: Analytic (kv.py) versus measured HF cache bytes on reduced-depth random-init instances, log–log; the dashed line is equality.

decompressed, sliding windows may be allocated in full, and recurrent states carry padding. RQ4 quantifies the gap on reduced-depth instances: 6 of 7 validation rows agree within 1%, spanning 6 attention classes, and the largest residual comes from a pure recurrent model whose state accounting differs (Table 5). The validation covers classes, not every census family.

Capacity model idealizations. The capacity model is deliberately minimal. It ignores allocator fragmentation — the problem paged attention was built to solve [17] — plus activation memory, kernel workspaces, and other runtime internals. Runtime overhead enters as a modeled parameter reported as a sensitivity grid (0–12 GiB; headline tables use 6 GiB), not as a measurement. Budgets are treated as uniform unified memory; prefill/decode disaggregation changes the arithmetic [33]. Finally, the Hub is a biased frame. We include original publishers only (33 organizations), open weights only, and no closed models; the fine-tune and merge ecosystem is excluded by design, and recent release years skew toward many small checkpoints. Fleet medians therefore describe the open-weight release stream, not production traffic, and readers should not read them as market shares.

7 Conclusion

We took a census of 2331 open-weight checkpoints from 33 organizations, reduced to 2068 family-level groups (292 distinct attention geometries), accounted their KV caches from configurations alone, and validated the accounting against reference implementations. Four verdicts. For RQ1, per-parameter KV cost is falling: the pooled 2025–2026 median is 0.413 of the 2023 median on the pre-registered unit and 0.071 under geometry deduplication (H1 supported on both), though not monotonically under the family unit, because the 2025 release mix and re-released same-geometry fine-tunes moved against the trend; excluding multimodal checkpoints flips the verdict (1.061), so the finding is real but fragile. For RQ2, parameter count is a weak predictor of KV footprint ($R^2 = 0.398$) while attention geometry alone explains 0.792, and MoE models carry 0.018 times the dense per-parameter KV cost (H2 and H3 supported). For RQ3, the KV-parity context grew 2.2 under the pre-registered unit and 11.0 under geometry deduplication (H4 refuted on the first, supported on the second): KV savings and weight growth offset each other, and release mix sets the median, even as KV-dominance at 32k context receded. For RQ4, the analytic accounting matches reference implementations on 6 of 7 validation rows within 1%, across 6 attention classes.

The practical rule is the title: size serving memory by attention geometry, not by parameter count. Read

Table 5: RQ4 validation: analytic (kv.py) vs measured HF cache bytes on reduced-depth random-init instances.

model	class	analytic	measured	rel.err	note
phi-2	mha	41.94M	41.94M	0.000%	MHA reference: 32 q/kv heads, head_dim 80 (hidden 2560 / 32), full attention. DynamicCache stores K and V of shape [B, kv_heads, T, head_dim] in float32 (layer 0: 2048 tokens); mat
Qwen2.5-0.5B	gqa	2.10M	2.10M	0.000%	GQA reference: 14 q heads / 2 kv heads, head_dim 64 (explicit in config). DynamicCache stores K and V of shape [B, kv_heads, T, head_dim] in float32 (layer 0: 2048 tokens); matches
starcoder2-3b	gqa+swa	8.39M	8.39M	0.024%	sliding-window GQA: all layers windowed, W=4096 (config sliding_window; no layer_types). T=4352 > W so the window saturates. transformers 5.x DynamicSlidingWindowLayer keeps W-1=40
gpt-oss-20b	gqa+swa	8.91M	8.91M	0.046%	window+global hybrid (layer pattern [sliding, full], period 2), W=128, T=2048 > W; MoE experts 32 -> 8 (routing only, no KV effect). sliding layer 0 stores W-1=127 of 2048 position
DeepSeek-V2-Lite	mla	9.44M	9.44M	0.000%	MLA: kv_lora_rank 512 + qk_rope_head_dim 64; 2 layers kept = layer 0 (dense MLP, first_k_dense_replace=1) + 1 MoE layer; routed experts 64 -> 8 (no KV effect). transformers caches
granite-4.0-h-micro	hybrid-mamba	27.73M	27.89M	0.565%	hybrid mamba2+attention: attention at layer idx 5, 15, 25, 35 -> period 10 (9 mamba2 + 1 GQA-attention layer kept). mamba2 layers hold constant state: conv_states [B,4352,4] (d_inn
mamba-2.8b	mamba	0.39M	0.41M	5.263%	pure recurrent (Mamba-1 SSM); state is T-independent, T=512 suffices. conv_states [B,5120,4] + recurrent_states [B,5120,16] in float32, both independent of T. The >1% gap is conv-s

the layer count, KV heads, head dimension, window sizes, and state shapes from the configuration; compute the cache; then pick the quantization levers the geometry leaves room for. On a fixed unified-memory device this computation, not the parameter count on the model card, decides how many long contexts the hardware can hold.

Three extensions follow. Measure real runtimes at scale, adding the allocator and kernel effects the analytic model omits. Extend to closed models where configuration metadata becomes available. Re-run the census periodically: window share, latent compression, and hybrid recurrence are still moving quickly, and the 2026 medians already rest on a partial year.

AI-Use Disclosure

This research was produced end to end by AI systems. At the author's request, an AI assistant (Claude, Anthropic) proposed the research question and wrote the research brief (research questions, hypotheses, inclusion criteria), the rules the agents operated under, and the external completion gate that checks the outputs. An autonomous agent swarm (Prime Agent orchestration, GLM-5.3 models) then executed the work inside a sandboxed repository: the census code, the data collection, the analysis, and the drafting of this paper. The author commissioned the study, provided the compute and the sandbox, and read the full manuscript and checked its claims against the generated results. The author takes full responsibility for the contents of this paper.

Two mechanisms stand in for manual verification. First, every reported result number traces to macros and tables generated by a deterministic, offline analysis pipeline (design constants and calendar years appear as literals); the completion gate re-runs that pipeline and requires byte-identical outputs. Second, every citation resolves to bibliographic metadata retrieved from the arXiv, OpenAlex, and Crossref APIs and

cached in the repository; no reference was added from memory. The analysis plan was registered before trend results were inspected, and deviations from it are recorded in the plan file with reasons. Figures and tables are regenerated by the same pipeline, and refuted hypotheses are reported as refuted. The repository at https://github.com/ozkangrk/kv-census ships the raw snapshots, the pipeline, and the generated artifacts, so every claim in this paper can be recomputed from source. We state the provenance plainly so that readers can weight the findings accordingly.

References

[1] Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J. Hewett, et al. Phi-4 Technical Report, 2024.

[2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, 2023.

[3] Chi-Chih Chang, Wei-Cheng Lin, Chien-Yu Lin, Chong-Yan Chen, Yu-Fang Hu, Pei-Shuo Wang, Ning-Chi Huang, Luis Ceze, et al. Palu: Compressing KV-Cache with Low-Rank Projection, 2024.

[4] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning, 2023.

[5] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, 2022.

[6] Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, et al. Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models, 2024.

[7] DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, et al. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, 2024.

[8] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale, 2022.

[9] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers, 2022.

[10] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, et al. Gemma 3 Technical Report, 2025.

[11] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, et al. The Llama 3 Herd of Models, 2024.

[12] Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces, 2023.

[13] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization, 2024.

[14] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, et al. Mistral 7B, 2023.

[15] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, et al. Mixtral of Experts, 2024.

[16] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention, 2020.

[17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention, 2023.

[18] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, et al. Jamba: A Hybrid Transformer-Mamba Language Model, 2024.

[19] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache, 2024.

[20] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, et al. RWKV: Reinventing RNNs for the Transformer Era, 2023.

[21] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient Context Window Extension of Large Language Models, 2023.

[22] Varun Rajesh, Om Jodhpurkar, Pooja Anbuselvan, Mantinder Singh, Ashok Jallepali, Shantanu Godbole, Pradeep Kumar Sharma, and Hritvik Shrivastava. Production-Grade Local LLM Inference on Apple Silicon: A Comparative Study of MLX, MLC-LLM, Ollama, llama.cpp, and PyTorch MPS, 2025.

[23] Noam Shazeer. Fast Transformer Decoding: One Write-Head is All You Need, 2019.

[24] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding, 2021.

[25] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive Network: A Successor to Transformer for Large Language Models, 2023.

[26] Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, et al. You Only Cache Once: Decoder-Decoder Architectures for Language Models, 2024.

[27] Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, et al. 2 OLMo 2 Furious, 2024.

[28] Haoyi Wu and Kewei Tu. Layer-Condensed KV Cache for Efficient Inference of Large Language Models, 2024.

[29] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient Streaming Language Models with Attention Sinks, 2023.

[30] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, et al. Qwen3 Technical Report, 2025.

[31] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated Delta Networks: Improving Mamba2 with Delta Rule, 2024.

[32] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, et al. H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models, 2023.

[33] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving, 2024.

[34] Jingwei Zuo, Maksim Velikanov, Ilyas Chahed, Younes Belkada, Dhia Eddine Rhayem, Guillaume Kunsch, Hakim Hacid, Hamza Yous, et al. Falcon-H1: A Family of Hybrid-Head Language Models Redefining Efficiency and Performance, 2025.